Basic Graph Theory

KNU Math 420

Classnotes

These notes are for a undergrad course in Graph theory based on Douglas West's 'Introduction to Graph Theory', which is referred to as 'the text'. Students should have the text. Test problems will be based on the problems in the notes. No only are the lists of problems at the end of each section problem from the notes, then problems written out in the notes are usually asking you to fill in details from the text. Some, but not all, of these problems will be answered in class.

Students should be familiar with the set theory and logic seen in Discrete Math.

v. November 4, 2026

1 Fundamental Concepts

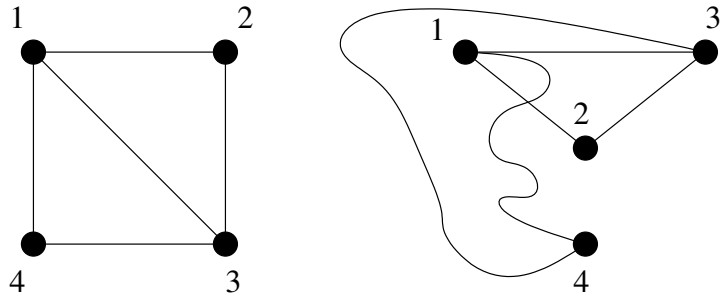
1.1 What is a graph?

A graph G consists of a non-empty finite set $V = V(G)$ of elements called *vertices* and a relation $E = E(G)$ on V, called the *edge* relation, which relate pairs of (not necessarily distinct) vertices. When edge relation is irreflexive and symmetric, the graph is called *simple*. Usually our graphs are simple. If $\{u, v\} \in E$, we say they u and v are *adjacent*, or that they are *neighbours*, and that they are the *end points of the edge* $\{u, v\}$. If $\{v, v\}$ is an edge, then it is a *loop*. Simple graphs don't have loops. If E is not symmetric, we call G a *digraph*.

For example

$$G = (\{1, 2, 3, 4\}, \{\{1, 2\}, \{2, 3\}, \{3, 4\}, \{4, 1\}, \{1, 3\}\})$$

is a graph. It has four vertices and five edges. As this mess of brackets gets troublesome to write, we drop them when it doesn't confuse things an write an edge $\{x, y\}$ as xy. The graph G can be represented pictorially as



Matrices and Isomorphism

To describe a particular graph we have to describe its vertices and its edges. The names of the vertices are usually not important, and so we usually only talk of a graph up to isomorphism.

Definition 1.1.1. An *isomorphism* is a bijective map $f : V(G) \to V(H)$ such that

$$uv \in E(G) \iff f(u)f(v) \in E(H).$$

If there is an isomorphism f from G to H, we say G and H are *isomorphic*, and write $G \cong H$.

The relation of being isomorphic is clearly an equivalence relation on the set of all graphs.

As we usually consider graphs to be the same if they are isomorphic. Mostly, graph theory is the study of *graph properties*– properties of a graph that are invariant under isomorphism. The number of vertices, and the size of the smallest cycle in the graph are such properties.

Occasionally, one does talk of *labelled graphs*, distinguishing graphs in the same isomorphism class, as it can help make things easier to count.

Problem 1.1.1. How many graphs are there on 4 vertices? How many non-isomorphic (unlabelled) graphs?

Obviously we will not not always be able to describe a graph with a picture. To put a graph into a computer, we often represent it as a matrix.

For a simple graph G on the vertex set $V = \{v_1, \ldots, v_n\}$ and having edgeset $E = \{e_1, \ldots e_m\}$ the *adjacency matrix* $A(G)$ is the symmetric $n \times n$ matrix $A = [a_{ij}]$ where

$$a_{ij} = \begin{cases} 1 & \text{if } v_i \sim v_j \\ 0 & \text{otherwise .} \end{cases}$$

The incidence matrix $M(G)$ is the $m \times n$ matice $M = [m_{ij}]$ such that

$$m_{ij} = \begin{cases} 1 & \text{if } v_j \in e_i \\ 0 & \text{otherwise .} \end{cases}$$

Problem 1.1.2. Where G is the four vertex graph drawn above, give $A(G)$ and $M(G)$.

Representing graphs this way often makes it easier to decide if they are isomorphic, but deciding if two graphs are isomorphic is a computationally hard problem.

Graphs as Models

See Section 1.1 of the text for several typical examples of things that are modelled by graphs. The following are typical questions we might ask about a graph. Try to answer them for the graph P shown in Figure 1. The definitions you need to understand these questions come later, but they are probably what you think they are.

- How many colours do we need, when colouring the vertices of P to be sure that adjacent vertices get different colours.
- Can you draw P without intersecting edges?
- How many edges (vertices) must you remove to *disconnect* P?

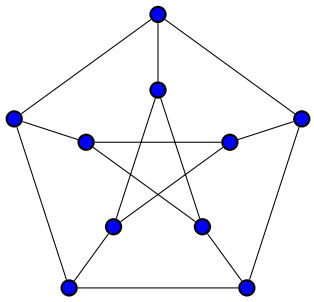
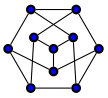


Figure 1: The Petersen Graph P

- For any two pairs of vertices u, u' and v, v' can you find a *path* between u and u', and another between v and v', such that the two are disjoint?
- What is the smallest *cycle* in P?
- How many 5-cycles are there in P?
- Can we partition the edges of P into disjoint 5-cycles?
- Can we find a *walk* in P that visits every vertex (edge) exactly once?
- Is P isomorphic to this graph? 

Some Common Graphs

We generally now consider graphs to be the same if they are isomorphic, and take blatent liberties with the names of their vertices. As such, we define some common graphs, these names apply to any graph that is isomorphic.

- The *complete graph*, or *clique* on n vertices, K_n, has $V(K_n) = \{v_1, v_2, \dots v_n\}$ and $E(K_n) = \{v_i v_j \mid 1 \leq i < j \leq n\}$.
- The *n-cycle*, C_n, has $V(C_n) = \{v_1, v_2, \dots v_n\}$ and $E(C_n) = \{v_i v_{i+1} \mid i = 1, \dots, n-1\} \cup \{v_n v_1\}$.
- The *n-path*, P_n, has $V(P_n) = \{v_1, \dots v_n\}$ and $E(P_n) = \{v_i v_{i+1} \mid i = 1, \dots, n-1\}$. (Many books use P_n to mean the path with n edges. This is actually the more common usage these days. Be careful of this when you are looking at other sources.)
- The *complete bipartite graph*, or *biclique* $K_{m,n}$, has $V(K_{m,n}) = \{a_1, \dots a_m\} \cup \{b_1, \dots, b_n\}$ and $E(K_{m,n}) = \{a_i b_j \mid i \in [m], j \in [n]\}$.

- The *n-cube*, Q_n, $V(Q_n)$ is the set of binary strings of length n, vertices u and v are adjacent if they differ in exactly one coordinate.

Subgraphs

Given a graph G, another graph G' is a *subgraph* of G, written $G' \leq G$ if $V(G') \subset V(G)$ and $E(G') \subset E(G)$.

A subgraph G' of G is a *proper* subgraph of G if $E(G')$ is a proper subset of $E(G)$. It is *spanning* if $V(G') = V(G)$, or *induced* if $E(G') = \{uv \in E(G) | u, v \in V(G')\}$. For a subset $T \subset V(G)$ the subgraph $G[T]$ of G *induced* on T is the induced subgraph of G with vertex set T; we get it from G by removing all vertices not in T.

Problem 1.1.3. How many non-labelled subgraphs are there of K_4? How many are proper? ...induced? ...spanning? ...spanning, induced and proper?

A subgraph of G isomorphic to a path (cycle, clique) is called a path (cycle, clique) of G. The *complement* $\overline{G}$ of a (simple) graph G is the graph with the same set of vertices and edgeset

$$E(\overline{G}) = \{uv \mid u \neq v \in V(G)\}.$$

An independent set of G is a clique of $\overline{G}$.

A graph G is *bipartite* if it is a subgraph of a biclique.

Problem 1.1.4. Show that the following properties are equivalent for a graph G.

i. G is bipartite.

ii. G there is a partition of $V(B)$ into sets A and B such that every edge has one endpoint in A and one in B.

iii. G contains no odd cycles.

(Showing that every non-bipartite graph has an odd cycle is non-trivial.)

Problem 1.1.5. Show that a subgraph G' of G is induced if and only if $\overline{G'}$ is a subgraph of $\overline{G}$.

Solution

Let $G' \subset G$ be induced. Clearly $V(\overline{G'}) \subset V(\overline{G})$, so we have to show that $E(\overline{G'}) \subset E(\overline{G})$. Let $e \in E(\overline{G'})$ then $e \notin E(G')$ so $e \notin E(G)$. So $e \in E(\overline{G})$.
On the other hand, let $\overline{G'} \leq \overline{G}$. We want to show that G' is induced. For $u, v \in V(G')$, with $uv \in E(G)$ we want to show that $uv \in E(G')$. Well $uv \in E(G)$ implies that $uv \notin E(\overline{G})$ so not in $E(\overline{G'})$, so because $u, v \in V(G')$, $uv \in E(G)$.

Decompositions and Special graphs

A graph is *self-complementary* if $G \cong \overline{G}$. Can you think of a self-complemenary graph? How many edges must it have? A self complementary subgraph H of K_n yields a decomposition of K_n into two copies of H.

A family of subgraphs of G is a *decomposition* of G if the edgesets of the subgraphs partition the edgeset of G.

Problem 1.1.6. Show that K_7 has a decompostion into copies of K_3. Does K_6?

The graph P from Figure 1 is called the Petersen graph. It comes up a lot. It has a neat construction that leads to its symmetry. Its vertex set can be viewed as the family of two-element subsets of the set $[5] = \{1, 2, 3, 4, 5\}$. Two vertices are adjacent if the subsets are disjoint.

This construction is useful in proving things about the Petersen graph. It has no triangles because this would require three mutually disjoint two elmenet subsets in [5]; which is impossible. It has no 4-cycles because any two non-adjacent vertices must, as sets, share one element, and so there is exactly one vertex adjacent to both of them. Thus the smallest cycle in P is a C_5. Can you find a 5 cycle? Can you show that any edge of P is in a 5-cycle?

The *girth* of a graph G is the length of its shortest cycle. An automorphism of a graph G is an isomorphism $f : G \to G$. A graph is *vertex transitive* if for every pair u, v of vertices, there is an automorphism f of G such that $f(u) = v$.

Problem 1.1.7. Show that the Petersen graph is vertex transitive.

The *degree* $d(v) = d_G(v)$ of a vertex v of a graph G is the number of edges of G that contain v (loops containing v are counted twice).

Problems from the Text

Section 1.1: 1 - 10, 12, 13, 14, 18, 23, 25, 41

1.2 Paths, cycles, and trails

Sept 11

Let x and y be vertices in a graph G. An *xy-walk* in G is a finite sequence

$$x = x_1 x_2 \dots x_{n-1} x_n = y$$

of vertices such that $x_{i-1} \sim x_i$ for $i = 1, \dots n$. The *length* of the walk is $n - 1$, (we count edges, not vertices). If the edges $x_i x_{i+1}$ of the walk are distinct, it is a *trail*, and if the vertices are distinct, it is a *path*. It is closed if $x_1 = x_n$.

Theorem 1.2.1. *Any xy-trail in a graph has an xy-path as a subgraph.*

Intuitively, we just start with an xy-trail and when we see a vertex repeated, we 'chop' out the bit between its appearences. A lot of times it is hard to write the intuitive proof rigorously. The text uses an induction on the length of the trail. We do the same proof, using the suggestion in exercise 13b.

Proof. Let $T = x_0x_1 \dots x_d$ be the shortest xy-trail in G; one exists by assumption.

We claim that T is an xy-path. If not, there are some $0 \leq i < j \leq d$ such that $x_i = x_j$. But then

$$x_0, x_1, \dots, x_{i-1}, x_i, x_{j+1}, \dots, x_d$$

is a shorter xy-trail T, which contradicts our choice of T. □

Problem 1.2.1. Prove Theorem 1.2.1 using induction.

A walk $x_1, \dots, x_n$ is *closed* if $x_1 = x_n$. A closed trail is a *circuit*. Prove the following with the same idea. When you see a vertex repeated, you can chop the circuit into two circuits. One of them must be odd.

Problem 1.2.2. Show that any circuit in a graph has a cycle as a subgraph. Show that any odd circuit in a graph has an odd cycle as a subgraph.

A graph G is *connected* if for every pair u, v of vertices there is a uv-path in G. A maximal connected subgraph of G is a *component*. A component is *trivial* if it contains only one vertex, the vertex is an *isolated* vertex. The *distance* $d(u, v)$ between two vertices of G is the length of the shortest path between them; it is infinite if they are in different components. A *cut-edge* or *cut-vertex* of a graph is a edge or vertex whose removal increases the number of components of the graph.

Theorem 1.2.2. *An edge is a cut-edge if and only if it is not in any cycles.*

Proof. See text. □

1.2.1 Bipartite Graphs

Recall that a graph G is bipartite if there is a partition $V(G) = A \cup B$ of its vertex set such that all edges are between A and B. We call A and B its bipartition. It can have more than one, but only if it is disconnected.

Sep 14

We asked you to show this earlier.

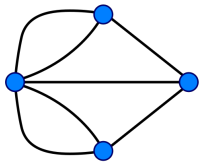
Theorem 1.2.3 (König (1936))**.** *A graph is bipartite if and only if it has no odd cycles.*

Proof. See text. □

1.2.2 Eulerian Circuits

It is often told that the first problem of graph theory was the problem of the Bridges of Königsberg. In the fictional city of Königsberg (Problem: prove or disprove my unpopular claim that Königsberg is fictional.) there were seven bridges. The dandies of the city, on slow Sunday, would walk about the town, playing a riddlish game. Starting whereever they pleased, they were to walk around, crossing every bridge, without crossing any bridge more then once. This game went on for centuries, and several liars claimed to have done it. Fermat once wrote in the margin of a book that it was simple enough. Until eventually, Euler proved it was impossible, and stabbed Fermat in a dual. (Prove that any of this is true.)

The city of Königsburg consisted of four dots, connected by seven bridges. It looked a lot like this:



A closed trail is called a circuit. A circuit of a graph G is an *Eulerian circuit* if it contains all edges and vertices of G. A graph is *Eulerian* if it contains an Eulerian circuit. A graph is *even* if all vertices have even degree.

The following is known as Euler's Theorem.

Theorem 1.2.4. *A graph is Eulerian if and only if it is even and connected.*

Proof. That an Eulerian graph must be connected is clear. That it must be even is almost as clear– let $C = v_1 v_2 \dots v_N$ be an Eulerian circuit. For any vertex v, count the number d of times that v occurs in $v_1 v_2 \dots v_{N-1}$. For each occurence $v = v_i$ is in two edges in C: $v_{i-1}v$ and vv_{i+1}. So v is in $2d$ edges in C. But C is contains each edge of G once, so v is in $2d$ edges in G.

Now, in the other direction, assume that G is a connected even graph. Our proof is by induction on the number m of edges. When $m = 0$, the fact that G is connected means that it has only one vertex, and this vertex is a closed trail containing all edge.

Assume then then $m > 0$. As it is even and non-empty it must contain a circuit C– indeed pick a vertex v_0 with an edge, start walking, whenever we enter a vertex other than v_0, we have visited an odd number of that vertices edges, so one remain to leave it by. We can keep walking then, until we get back to v_0.

Now, the graph $G' = G - C$ need not be connected, but each component is even, so contains a Eulerian circuit of the component by induction. Moreover

C visits each component, as G is connected. Traversing C, the first time we encounter a vertex of a component X, we pause our traversal of C, traverse the Eulerian circuit of X which brings us back to where we entered X, and then continue our traversal of C.

This yields an Eulerian circuit of G, as needed. □

Problem 1.2.3. Using Euler's Theorem, show that in a connected graph G with $2k$ vertices of odd degree, there is a decomposition of G into k trails, and no fewer.

Problems from the Text

Section 1.2: 1-12, 14, 16, 17, 27, 29, 43

1.3 Vertex Degrees and Counting

Sep 18

The *size* of a graph $G = (V, E)$ is the number $e(G) = |E|$ of edges it has, while the *order* is the number $n(G) = |V|$ of vertices it has. Quite often we use n on its own for the order of the graph we are talking about, and m for its size. (We can't use e, because we usually use that for an edge of G.) Or we will specify n by calling G an n-vertex graph.

For a vertex v of G, the *neighbourhood* $N(v) = N_G(v)$ of v in G is the set of edges in $E(G)$ that contain v. So for a simple graph, the degree of a vertex is $d(v) = |N(v)|$. The *min-degree* $\delta(G)$ of G is the minimum of the degrees of its vertices, and the *max-degree* $\Delta(G)$ is the maximum. If all vertices of G have degree k, then G is *k-regular*.

While many of the 'common graphs' we have seen are regular, it is a very rare property for a graph to be regular.

Problem 1.3.1. What is the size of a k-regular graph of order n?

A vertex transitive graph is, of course, regular. We showed earlier that the Petersen graph is vertex transitive, so is clearly 3-regular.

This example, from Section 1.3 of the text, shows how nice it can be to have vertex transitivity. A *claw* is the graph $K_{1,3}$.

Example 1.3.1. We show that the Petersen graph P has ten 6-cycles, by showing that there is a 1-to-1 bijection between 6-cycles and claws. As each of the 10 vertices of P is the center of exactly one claw, P has ten claws, so this is enough.

Pick the claw in P with center v, and leaves a, b, c. Each of the leaves has two other edge, but as the leaves are distance 2 apart, non of their edges other than the ones to v, can share an endpoint, as P has no 3-cycles. So when we

remove a claw, the six edges from the claw to the rest of P, all go to different vertices of P. Thus what is left is a 2-regular graph of order 6. As P has no triangles, this can only be a 6-cycle.

Not all regular graphs are as nice as the Petersen graph.

Problem 1.3.2. Find a regular graph that is not vertex transitive.

But regularity can be still be nice. There is a difficult open conjecture. The *deck* of a graph G is the multiset of *unlabelled* vertex deleted subgraphs

$$\{G \setminus v_i \mid v_i \in V(G)\}.$$

The following conjecture posits that no two non-isomorphic graphs with at least 3 vertices can have the same deck.

Conjecture 1.3.2 (Graph Reconstruction Conjecture (Kelly and Ulam 1942))**.** *A graph of order at least* 3 *is uniquely determined by its deck.*

Problem 1.3.3. Why do we need three vertices to conjecture this?

This conjecture is almost 100 years old, and we cannot prove it. But we do know that it is true for certain classes of graphs. In the problems you will show the following; it is quite easy– maybe you see already why it is true.

Fact 1.3.3. *The Graph Reconconstruction Conjecture is true for regular graphs.*

Extremal graphs

Problems in extremal graph theory ask such things as how big or small a graph can be and have a certain property.

Some simple such questions are:

Problem 1.3.4. What is the maximum number of edges an n-vertex graph can have? What is the minimum?

Maybe this problem was too simple. This one is only marginally harder.

Problem 1.3.5. What is the maximun number of edge in a **disconnected** graph on n vertices?

Still too simple? Maybe, but it introduces some ideas. To answer this question, you found an extremal graph, a disconnected graph on n vertices with _____ edges. This is called the *extremal graph* for the problem. Then you had to prove that there was no graph on n vertices with more edges.

There are many extremal graphs for the following question.

Problem 1.3.6. What is the minimum number of edges in a **connected** n-vertex graph?

The following problem is maybe not an extremal problem, but has the flavour of one, and is a useful tool for may extremal problems.

Theorem 1.3.4. *Every simple graph has a bipartite subgraph containing at least half of its edges.*

Proof. The proof is by induction on the number, n, of vertices of the graph G. The statment is clearly true when $n = 1$, so assume that $n \geq 2$ and that the result holds on all graphs of order less then n. Let v be a vertex of G and let $G' = G \setminus v$. By the induction hypothesis, there is a partition $V(G') = A' \cup B'$ such that at least half of the edges of G' are between A' and B'. We may assume that at least half of the edges of G that contain v are to vertices in B', and so setting $A = A' \cup \{v\}$ we get a partition $V(G) = A \cup B'$ such that at least half of the edges of G are between A and B'. □

A graph G is H-free if it contains no *induced* copy of H. Mantel proved one of the first results of extremal graph theory. We will later see a generalisation by Turan.

Problem 1.3.7. How many edges can a graph on n vertices have, and have no triangles?

There is a pretty good family of candidates for the extremal graph– graphs with lots of edges an no triangles: the bipartite graph $K_{a,(n-a)}$ has no triangles and has $a(n-a)$ edges. You can use calculus, or an argument as we used in the above theorem, to do the follwing.

Problem 1.3.8. Show that $a(n-a)$ is maximized when $a = \lfloor n/2 \rfloor$, and so show that there is a bipartite graph of order n with $\lfloor n/2 \rfloor \lceil n/2 \rceil = \lfloor n^2/4 \rfloor$ edges.

So a graph can have $\lfloor n^2/4 \rfloor$ edges and have no triangles. Can it have more?

Definition 1.3.5. Let $T(n)$ be the maximum number of edges in a triangle free n-vertex graph.

The above example showed that $T(n) \geq \lfloor n^2/4 \rfloor$.

Theorem 1.3.6 (Mantel (1907))**.** *For all $n \geq 1$ we have $T(n) \leq \lfloor n^2/4 \rfloor$.*

Proof. As $T(n)$ is an integer, it is enough to show that $T(n) \leq n^2/4$. The base case is $n = 1$ and 2 for which it is clearly true. Assume the theorem holds for all graphs on at least n vertices, and let $G = (V, E)$ be a triangle free graph on $n + 2$ vertices.

Choose an edge $e = \{x, y\}$ from E, and let $G' = (V', E')$ be the subgraph of G induced by $V' = V \setminus \{x, y\}$. Let E_x be the set of edges from x to a vertex of V', and E_y be the set of edges from y to a vertex of V'.

As $E = E' + E_x + E_y + \{e\}$, and from the fact that G is triangle-free we get that $|E_x| + |E_y| \leq n$. Thus from the induction hypothesis we get that that

$$|E| \leq \frac{n^2}{4} + n + 1 = \frac{(n+2)^2}{4}.$$

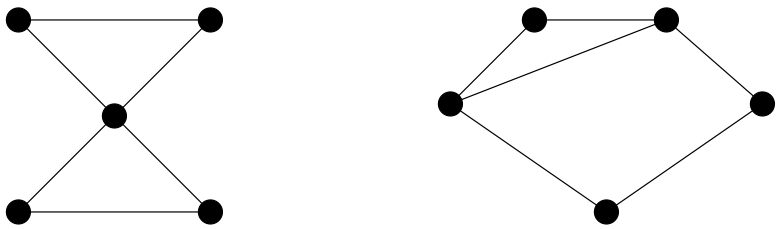
□

Graphic sequences

Definition 1.3.7. The *degree sequence* of a graph G is the multiset of the degrees of the vertices of G. The minimum degree, $\delta(G)$, and maximum degree, $\Delta(G)$, of a graph G are the minimum and maximum degrees respectively in its degree sequence.

Problem 1.3.9. Show that every graph contains a path of length at least $\delta(G)$.

Clearly, degree sequence, minimum degree, and maximum degrees are graph properties. So the following two graphs, which have the same number of vertices, and edges are not isomorphic as they have different degree sequences: $(4, 2, 2, 2, 2)$ and $(3, 3, 2, 2, 2)$ respectively, (and in fact different maximum degrees.)



Here is a simple observation about degree sequence:

Theorem 1.3.8. *If $(d_1, d_2, \ldots, d_n)$ is the degree sequence of a graph, then the degree sum: $\sum_{i=1}^{n} d_i$ is equal to $2|E(G)|$.*

Proof. Each edge of the graph contributes 2 to the degree sum, one for each endpoint. □

Corollary 1.3.9. *In any graph G, there is an even number of vertices of odd degree.*

Problem 1.3.10. What is the degree sequence of the cube Q_n?

Clearly isomorphic graphs have the same degree sequence. But the converse is clearly not true.

Problem 1.3.11. Find graphs with the same degree sequence that are not isomorphic.

A sequence $D = (d_1, d_2, \ldots, d_n)$ non-increasing integers is *graphic* if there is a graph G on the vertices $v_1, \ldots, v_n$ such that $d(v_i) = d_i$ for all $i = 1, \ldots, n$. Such a graph G is called a *realisation* of D. We saw above that one necessary condition for a sequence to be graphic is that it sums to an even number. But this is not sufficient: $(4, 2, 0)$ is not graphic.

There is a recursive method to decide if a given sequence is graphic.

Lemma 1.3.10. *(Havel-Hakimi) If a sequence $D = (d_1, d_2, \ldots, d_n)$ is graphic, then there is a graph with degree seqence D such that the maximum degree vertex v_1 is adjacent to the d_1 vertices of next highest degree.*

Before we prove this. Let's see how it allows us to decide if a sequence is graphic. If D is graphic, then it has a realisation as in the lemma. If we remove the edges from v_1 then the graph we get has degree sequence

$$D' = (0, d_2, d_3 - 1, \ldots, d_{d_1+1} - 1, d_{d_1+2}, \ldots, d_{n-1}, d_n)$$

which we get by changing d_1 to 0 and reducing the next d_n values by one.

On the other hand, if the sequence D' is graphic, then so is D.

This gives us a recursive algorithm for deciding if a degree sequence D is graphic: keep reducing the sequence as above until all the degrees are reduced to 0 or less. This clearly terminates, as the sum of degrees is reduced at each step.

If we end with a sequence of zeros, D was graphic. If we get any negative numbers, it was not graphic.

(Sometimes this is done reordring the sequences at each step, but it is useful to keep them unordered, as it is them easier to reconstruct the graph if the sequence is graphic.)

Sep 21

Problem 1.3.12. Use this algorithm to decide if $(3, 3, 2, 2, 2)$ is graphic.

Now let's prove the lemma.

Proof. Let $D = (d_1, d_2, \ldots, d_n)$ be a graphic sequence. Given a realisation G of D, let $T = \{v_2, \ldots, v_{\Delta+1}\}$, and let $t_G = |N(v_1) \cap T|$. As D is graphic, it

has some realisation. Let G be on that maximises t_G. If $t_G = \Delta$ then we are done, so we may assume, towards contradiction, that $t_G < \Delta$. We find a new realisation G' with $t_{G'} > t_G$, so the lemma follows by contradiction.

As $N(v_1) \neq T$, there is $x \in T$ with $v_1 \not\sim x$ and $y \notin T$ with $v_1 \sim y$. Furthermore, since $x \in T$ and $y \notin T \cup \{v_1\}$, $\mathrm{d}(x) \geq \mathrm{d}(y)$. So there is some vertex s with $s \sim x$ and $s \not\sim y$.

Removing the edges v_1y and xs from G and adding v_1x and ys we get a graph G' with the same degrees on every vertex, (so another realisation of D), but with $t_{G'} = |N_{G'}(v_1) \cap T| = |(N_G(v_1) \cap T) \cup \{x\}| = t_G + 1$, as needed for the contradiction. □

Problem 1.3.13. Decide, using Havel-Hakimi, whether or not the sequence $(5, 5, 5, 3, 2, 2, 1, 1)$ is graphic.

Our proof above used what we might call a 2-*switch*. Let a graph G have vertices $a, b, c, d \in V(G)$ such that

$$a \sim b \not\sim c \sim d \not\sim a.$$

The graph we get from G be removing the edges ab and cd, and adding the edges bc and da, is a 2-switch of G. The process of adding and removing the edges is also called a 2-*switch*.

Corollary 1.3.11 (Berge (1973))**.** *Let G and H be graphs with the same degree sequences. One can get from G to H be a sequence of 2-switches.*

Proof. By the above proof of Havel-Hakami, we can switch both graphs to the same graph. □

Problems from the Text

1.3: 1-8, 9, 10, 12, 13, 14, 21, 28, 30, 36, 37, 38, 39, 46, 62

We skip Section 1.4 on digraphs, I do it in discrete math, and don't want too much overlap.

2 Trees and Distance

2.1 Basic Properties

A *forest*, or *acyclic graph*, is a graph with no cycles as subgraphs. A *tree* is a connected forest. A *leaf* is a vertex of degree 1. A *spanning tree* of a graph G is a spanning subgraph which is a tree.

A lot of the results we give about trees might seem obvious. This, then is a good place to practice proper proof. For any of these results, I can draw a picture, and wave my hands, and convince you that they are true. But a picture is usually not a proof. And proving what seems clear is often quite difficult.

Problem 2.1.1. Show that a graph is connected if and only if it has a spanning tree.

Hand-waving proof: The spanning tree connects the graph, so a graph with a spanning tree is clearly connected. On the other hand, let G be connected and H be a minimal connected subgraph. We are done unless H has a cycle, but if it has a cycle we can remove some edge from the cycle without disconnecting H, contradicting the choice of H.

Everything I say there is true. And you believe it. But is it enough. Should I justify some of my statements? What does it mean that 'the spanning tree connects the graph'? Why does removing an edge from the cycle not disconnect H?

Problem 2.1.2. Show that a connected graph is a tree if and only if every edge is a cut-edge.

The following will be useful in inductions. This result requires the assumption, which we always make, that graphs are finite.

Lemma 2.1.1. *Any tree T of order at least 2 has at least two leaves, and removing a leaf from such a tree produces a smaller tree.*

Proof. Let T be a tree with at least two vertices. Let P be a maximal path in T. It is non-trivial because T has at least one edge. The endpoints of P are leaves, because if they have more than one neighbour in P, then we have a cycle, which is impossible in a tree, and if the have any neighbours not in P, then P was not maximal. Thus we have at least two leaves.

Let ℓ be a leaf and $T' = T - \ell$. T' is certainly acyclic, as T was. It is also connected, as ℓ, having degree 1 is not in the path in T between any two vertices u, v in $V(T) \setminus \{\ell\}$. □

As trees are so basic, having several characterisations of them can be useful in proofs.

Theorem 2.1.2. *Let $G = (V, E)$ be an n-vertex graph. Then the following are equivalent.*

i. G is connected and acyclic.

ii. G is connected and has $n - 1$ edges.

iii. G is acyclic and has $n - 1$ edges.

iv. Any two vertices of G are connected by a unique path.

Proof. The text breaks the proof down to proving the following five implications.

$$i \Rightarrow \{ii, iii\} \qquad ii \Rightarrow \{i, iii\} \qquad iii \Rightarrow \{i, ii\} \qquad i \Rightarrow iv \qquad iv \Rightarrow i$$

Sep 28

Problem 2.1.3. Pick one implication and write a proof for it. (In the text, the proofs of the second and third implications use that the first is already proved; this is allowed.) Don't look at the text until you have completed your proof.

□

Proposition 2.1.3. *Let T and T' be spanning trees of a connected graph G and let $e \in E(T) - E(T')$.*

- *There is an edge $e' \in E(T') - E(T)$ such that $T - e + e'$ is a spanning tree of G.*
- *There is an edge $e' \in E(T') - E(T)$ such that $T' + e - e'$ is a spanning tree of G.*

Proof. i) As T is a tree, so e is a cut-edge, $T - e$ has two components. There is a path in T' between a vertex u in the first and a vertex v in the second, so there is an edge e' of T' in that path going between the components. Thus $T - e + e'$ is connected, and so by the edge count, is a spanning tree.

ii) The graph $T' + e$ has one cycle, and any edge $e' \neq e$ in this cycle can be removed without disconnecting it, so $T' + e - e$ is connected, and so by the edge count is a spanning tree. □

This result has a cool application given at the end of the Section in the text.

Distance in trees and graphs

Recalling that the distance $d(u, v)$ between vertices u and v in a graph G is the length (number of edges) of the shortest uv-path in G, the *diameter* of G is

$$\operatorname{diam} G = \max_{u,v \in V(G)} d(u, v).$$

The *eccentricity* of a vertex u is

$$\epsilon(u) = \max_{v \in V(G)} d(u, v),$$

and the *radius* of G is

$$\operatorname{rad} G = \min_{u \in V(G)} \epsilon(u).$$

The *center* of G is the set of vertices of minimum eccentricty, or the subgraph of G induced by them.

Problem 2.1.4. Find the diameter and radius of K_n, P_n, C_n, Q_n and P.

Graphs with big radius tend to have few edges while graphs with small radius have many. One particular example of this is the following.

Problem 2.1.5. Show that if $\operatorname{diam}(G) \geq 3$ then $\operatorname{diam}(\overline{G}) \leq 3$.

Theorem 2.1.4 (Jordan (1869))**.** *The center of any tree is a vertex or an edge.*

Proof. See the book for the strong induction on n. □

Problems from the Text

2.1: 1-16, 18, 19, 32, 33, 38, 40, 51, 54

2.2 Spanning Trees and Enumeration

Oct 02

How many labelled spanning trees are there in K_5? How about K_n? This is a non-trivial question. Cayley gave us an answer.

Theorem 2.2.1 (Cayley's formula)**.** *There are n^{n-2} different labelled trees on n vertices.*

By labelled, we mean that the path $1 \sim 2 \sim 3$ on the vertex set $\{1, 2, 3\}$ is different from the path $2 \sim 1 \sim 3$. (But it is the same as the path $3 \sim 2 \sim 1$.)

There are several vastly different proofs of Cayley's Formula.

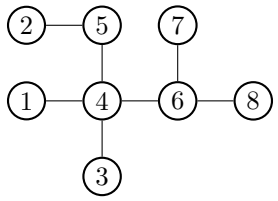
We see one by Prüffer. Observe that the number of $n-2$-element sequences $s \in [n]^{n-2}$ of integers from the set $[n] = \{1, 2, \ldots, n\}$ is exactly n^{n_2}. We show a correspondence between these sequences and the labelled trees on n vertices.

Prüffer Encoding Given a tree $T = T_1$ with vertices labelled with the integers $[n]$, construct $s = s(T) \in [n]^{n-2}$ as follows.

For $i = 1, \ldots, n-2$,

- let ℓ_i be the lowest labelled leaf in T_i.
- let s_i be the unique neighbour of ℓ_i in T_i.
- let $T_{i+1} = T_i \setminus \ell_i$.

Problem 2.2.1. Find the sequence $s(T)$ corresponding to the tree T:



It is clear that Prüffer Encoding encodes each tree T on $[n]$ to a code $s(T)$ in $[n]^{n-2}$. Moreover for any tree T, the leaves of T are exactly the vertices of T not listed in $s(T) = (s_1, \ldots, s_{n-2})$, and the first leaf of T removed, that is, the one with lowest label, is adjacent in T to s_1.

To prove Cayley's theorem we have to show that each code s is the code $s(T)$ of exactly one tree T.

Proof of Cayley's Formula. We prove, by induction that the construction $s \mapsto T(s)$ is a bijection from $[n]^{n-2}$ to the set of spanning trees on $[n]$. When $n = 2$ there is a single tree and a single empty string. As the construction is well defined, it must be a bijection. So assume that $n \geq 3$, and that the construction is a bijection for values smaller than n.

Fix a code $s = (s_1, \ldots, s_n) \in [n]^{n-2}$. Any tree T for which $s(T) = s$ has that its lowest labelled leaf $\ell \in [n]$ adjacent to s_1. For all such trees T, if we remove the leaf ℓ, we get a tree T' on $n-1$ vertices $[n] \setminus \ell$ with $s(T') = (s_2, \ldots, s_{n-2})$. By induction though, there is exactly one such tree T', and so exactly one tree T with $s(T) = s$. □

Some proofs of Cayley's formula via Prüffer Encoding find an inverse construction.

Problem 2.2.2. Find a construction that given a sequence $s = (s_1, \ldots, s_{n-2})$, constructs a tree $T = T(s)$ on the vertex set $[n]$ such that $s(T(s)) = s$ and $T(s(T)) = T$.

Observe also, with the Prüffer encoding $s = s(T)$ of a tre T, a vertex of degree d occurs exactly $d-1$ times in the string s– it appears when a neigbour is removed, and all are removed except the last one, when the vertex is a leaf. So the same proof gives the following. Recall the the sum of degrees on a tree on $[n]$ will be $2n-2$, and that $0! = 1$.

Corollary 2.2.2. *Given degrees $d_1, \ldots, d_n$ summing to $2n-2$, there are exactly*

$$\frac{(n-2)!}{\prod(d_i-1)!}$$

trees on $[n]$ in which the vertex i has degree d_i.

Proof. I guess you have to know how to count the number of permutations of a multiset. You would have seen this in discrete mathematics. Such trees are in bijection with the strings in $[n]^{n-2}$ consisting of $d_i - 1$ copies of the vertex i. If we view each copy of i as distinct, then there are $(n-2)!$ such strings, but this is overcounting, as the copies of i are not distinct. So we have to divide by the number $(d_i-1)!$ of permutations of the $d_i - 1$ different copies. Doing this for each i gives the formula. □

Spanning trees in other graphs

Oct 12

Cayley gave us a nice formula for counting the spanning trees in K_n. One would expect the problem to get harder for general graphs. And it does, but there is a nice formula using linear algebra. (This is in the text, and is not terribly hard, but a unhurried presentation of it always seems to take me two classes, which is a bit much for us. So we skip it.) There is also a recursive way to count these trees. This won't be efficient, but is perhaps useful for small graphs.

Let $e = uv$ be an edge of a graph G. The graph $G - e$, read "G less e", is the subgraph we get by removing the edge e. The *contraction* $G \cdot e$ is the graph we get by pushing the vertices u and v together into a new vertex x. Any neighbour of either u or v in G is a neighbour of x in $G \cdot e$.

A contraction makes loops and may make multiple edges. Depending on the application, we will keep or through these out. For our application to spanning trees, we keep the multiple edges this makes, but not the loops.

Here is the application. Let $\tau(G)$ denote the number of labelled spanning trees in a graph G.

Proposition 2.2.3. *For a graph G with an edge e,*

$$\tau(G) = \tau(G \setminus e) + \tau(G \cdot e).$$

Proof. Show that the spanning trees of G that do not contain e are counted by $\tau(G-e)$; while those that contain e are counted by $\tau(G \cdot e)$. □

Problem 2.2.3. Use the above proposition to count the number of spanning trees in the graph  . Example 2.2.9 in the text will help.

Decompositions and Graceful Labelings

The following conjecture has long been open. As of 2020, we know that it holds when n is really big. You should try to solve it with AI.

Conjecture 2.2.4 (Ringel (1964))**.** *For any tree T with m edges, K_{2m+1} (which has $m(2m+1)$ edges) can be decomposed into $2m+1$ copies of T.*

This is related to the following notion.

A *graceful labelling* of a graph G with m edges is a function $f : V(G) \to [m]_0 := \{0, 1, \ldots, m\}$ such that where $f^* : E(G) \to [m]_0$ is the induced edge labelling defined by $f^*(uv) = |f(u) - f(v)|$, we have

$$\{f^*(uv) : uv \in E(G)\} = \{1, \ldots, m\}.$$

In particular, adjacent vertices get different labels, so f^* actually maps to $[m]$.

Problem 2.2.4. Find a graceful labelling of P_4

Indeed, we have the following.

Theorem 2.2.5 (Rosa (1967))**.** *If a tree T with m edges has a graceful labeling, then K_{2m+1} has a decomposition into $2m+1$ copies of T.*

Proof. Let $f : V(T) \to [m]_0$ be a graceful labelling of T. One can view it as an ordering of the vertices of T. This defines a copy T_0 of T on the vertices $[m]_0 \subset [2m]_0 = V(K_{2m+1})$, the label $f^*(uv)$ on an edge uv is the distance in $[2m]_0$ between $f(u)$ and $f(v)$. Each distance d in $[m]$ occurs exactly once.

The ath copy of T_a of T in K_{2m+1} is the image of the injection

$$f_a : V(T) \to V(K_{2m+1}) : i \mapsto a + f(i).$$

So T_a is T_0 'rotated forward' a steps.

Over the $2m+1$ copies, each distance $d \in [m]$ occurs exactly $2m+1$ times, and each time with a different lower vertex. So the copies of T decompose the edges of K_{2m+1}. □

Problem 2.2.5. Use your graceful labelling of P_4 to get a P_4-decomposition of K_7.

It is known that paths, and stars have graceful labellings, and you will show in problem 31 that caterpillars do.

Problems from the Text

Section 2.2: 1, 2, 5, 7, 8, 10, 15, 20, 28 (See Def 2.2.17 of text for caterpillars), 30, 31

2.3 Optimization and Trees

A *weighted graph* $G = (G, w)$ consists of a graph G and a non-negative weight $w(e)$ for each edge in $E(G)$. The *weight* of a subgraph H of a weighted graph G is

$$w(H) = \sum_{e \in E(H)} w(e).$$

The following is a simple algorithm for finding the minimum weight spanning tree of a connected weighted graph.

Algorithm 2.3.1 (Kruskal's Algorithm)**.** ***Input***: A weighted graph (G, w).

Output: Edge-set T of a minimum weight spanning tree of G.

Initialise: Set $T = \emptyset$

Algorithm: Order the edges of G as $e_1, e_2, \ldots, e_m$ such that $w(e_i) \leq w(e_{i+1})$ for all i, and then for $i = 1, \ldots, m$ add an edge to T if doing so doesn't make any cycles.

This algorithm clearly terminates after $m < n^2$ steps. It is an example of a greedy algorithm– it does what looks best locally. Often such algorithms do not give an optimal solution, but in this case it does.

Theorem 2.3.2. *In a connected weighted graph, Kruskal's algorithm returns a minimum weighted spanning tree.*

Proof. It seems clear that the algorithm returns a tree T. By defininition, the graph it returns is acyclic. If it is not connected, then there is an edge of G between some two components. Adding this edge does not make a cycle, so would not have made a cycle when the algorithm considered adding it. So the algorithm should have added it.

Now, we show that the returned tree T has minimum weight over all spanning trees. Towards contradiction, assume that the returned T with edges $t_1, \ldots, t_{n-1}$ (such that $w(t_i) \leq w(t_{i+1})$ for all i) does not have minimum weight. Of the trees of weigh less than $w(T)$ let T^* be the that maximises the index i such that t_i is the first edge of T not in T^*. Adding t_i to T^* makes a cycle, and there is some edge t^* of $T^* \setminus T$ in this cycle. We have $w(t_i) \leq w(t^*)$ or Kruskal would have chosen t^* for T before choosing t_i. Thus the tree $T^* \cup \{t_i\} \setminus \{t^*\}$ also has

Oct 16

weight less than T, and agree with longer than T^* does, a contradiction of the choice of T^*. □

Note

When we calculate the running time of a graph algorithm, the data structure we use to represent the graph can make a difference. We don't want to go too deep into this, so we will pretend we have a black box that can tell us, in a single operation, if two vertices are adjacent; and it can iterate the neighbours of a vertex– this takes one operation for each neighbour. When we say an algorithm takes time t, it means it takes upto t operations.

The running time of this algorithm depends mostly on how quickly one can order the m edges. This depends on how the edges are stored, but we usually say it takes time $O(m \log m)$.

Problem 2.3.1. Algorithm 2.3.1 is high-level; not ready for implementing on a computer. It doesn't tell us how to decide if adding an edge to T makes a cycle. Write some more detailed code/pseudo-code for the algorithm that keeps track of the components of T as we build it, and uses this information to decide whether or not to add an edge.

Note

In applying algorithms, sometimes one can save 'time' by using more 'space' and this is sometimes considered. The time is always at least as much as the space, because to use the space we must write to it.

Breadth First Search and Dijkstra

A *rooted* tree is a tree T with a special vertex r. Orienting all the edges towards t, it has the defining property that the every vertex except r has a single out-arc. (An *arc* is an oriented edge.) The unique path from a vertex v in the tree to r is easy to find– you just follow the out-arcs.

A rooted spanning tree T of a graph G is a *BFS-tree* if the following holds for every vertex v:

$$d_T(r, v) = d_G(r, v).$$

Such trees are useful in finding the distance $d_G(r, v)$ of vertices from r. BFS-trees are so called because the are exactly the trees we can get via a breadth first search (BFS) of the graph.

In this algorithm we will build a set T of edges of a tree. To do so we keep track of the vertices in a set U of used vertices and a queue Q of active vertices.

A *queue* is a set ordered from a *head* to a *tail*. We will *enqueue* vertices, adding them at the tail, and *dequeue* vertices, removing them from the head.

Algorithm 2.3.3 (BFS Tree)**.**

Input: A connected graph G and a root vertex r.

Output: A spanning tree T.

Initialise: $T := \emptyset$, used vertices $U := \emptyset$, and queued vertices $Q := \{r\}$.

Algorithm: While $Q \neq \emptyset$:

- Dequeue a vertex u from Q and add it to U.
- Add to T all edges from u to neighbours v not in $U \cup Q$, and enqueue those neighbours in Q.

The running time of this algorithm is $O(m)$. Each edge must be observed. (It might seem like we only need the edges from u to vertices v not in $U \cup Q$, but we actually must check, for each neighbour of u, whether it is in $U \cup Q$.)

Often the goal of finding a BFS tree is to determine the distance from any vertex u to the root r. There are several common ways we do this.

i. When enqueuing a neighbour v of a vertex u record the distance $d(r, v) = d(r, u) + 1$, and return these distances with the output.

ii. Like above but return the partition of $V(G)$ into the levels

$$L_i = \{v \in V(G) \mid d(r, v) = i\}.$$

iii. Return T as *(rooted)* directed tree– when u enqueues a neighbour v direct the edge uv as $v \to u$. Doing so, every vertex by r has a single out arc. To find the distance from v to r, we follow the out-arcs to r, counting them to get the distance to r. The out-neighbour u of v is also called its *predecessor*. Sometimes we just return every vertex's predecessor.

Problem 2.3.2. Show by induction that all vertices that are distance d from r are enqueued before any vertex that is distance $d + 1$ from d is enqueued. Conclude that when a vertex v is enqueued, T contains a shortest (r, v)-path.

Oct 19

The algorithm changes a bit for a weighed graph (G, w) when we redefine the distance between r and v in G as the weighted distance. The *weighted distance* $\mathrm{wd}(u, v)$ between vertices of G is the mininum, over all (u, v)-paths P in G, of the weight $w(P)$. The algorithm is similar, only we replace the queue Q with an ordering of the vertices v by temporary distances $t(v)$ from v to r. This distance can be updated (decreased) until the vertex is added to U. When a vertex is added to U we add to T the edge to the vertex u already in u that defined its smallest temporary distance. We keep track of this u as its predecessor $p(v)$.

Algorithm 2.3.4 (Dijkstra)**.**

Input: A connected weighted graph (G, w) and a root vertex r.

Output: A spanning tree T rooted at r such the the weighted distance of vertices from r is the same as in G.

Initialise: $T := \emptyset$, $U := \emptyset$, $t(r) := 0$, $t(v) := \infty$ and $p(v) =$ undefined for all $v \neq r$.

Algorithm: While $U \neq V(G)$:

- Let u be a vertex not in U minimizing $t(u)$. Add u to U and add the edge $(u, p(u))$ to T.
- For each neighbour v of u not in U set $t(v) := \min(t(v), t(u) + w(uv))$, and if this decreases $t(v)$ then set $p(v) = u$.

Return: T and $\mathrm{wd}(r, v) = t(v)$ for all v.

The running time is $O(m^2)$ as we now have to scan through Q for a vertex with minimum $t(u)$. One can get $O(m \log m)$ by keeping Q ordered by $t(u)$.

As with the BFS, we might want to include the final values $t(v)$, the weight of the minimum weight vr-path in G, with the output.

Theorem 2.3.5. *The distance* $\mathrm{wd}(u)$ *returned for a vertex by Dijkstra is the weighted-distance from* u *to* r *in* G*.*

Proof. This is certainly true of r. Let u be a vertex of G different than r, and assume that the statement is true of all vertices added to U before u. When $t(u)$ is last reduced, it is set as

$$t(u) = t(p(u)) + w(p(u), u) = \mathrm{wd}(r, p(u)) + w(p(u), u)$$

so is at most $\mathrm{wd}(u, r)$. If the final value $t(u)$ is greater than $\mathrm{wd}(u, r)$ then there is some vertex p adjacent to u in a min weight path to r that is not added to U before u. Let p' be the furthest vertex from u in this path that was not added to U before u, and let p'' be its neighbour in the path furthest from u. So p'' is added to U before u. ... □

Problems from the Text

Section 2.3: 1-3, 6, 7, 11, 18, 21

3 Matchings and Factors

3.1 Matchings and Covers

A *matching* in a graph G is a set of edges, no two of which share a vertex. A vertex in an edge of a matching M is *saturated* if it is in one of the edges of M. A matching is *perfect* if every vertex is saturated.

Problem 3.1.1. How many perfect matchings does $K_{n,n}$ have?

A little harder.

Problem 3.1.2. How many perfect matchings does K_n have?

Maximum Matchings

A matching in G is *maximal* if it is contained in no bigger matching, and is *maximum* if it has the maximum number of edges over all matchings.

Given a matching M of G, a path P is *M-alternating* if its edges alternate between edges in M and edges not in M. It is *augmenting* if the endpoints are not saturated by M. If P is an M-augmenting path, one can get a new matching by removing the edges $E(P)$ from M, and adding the edges of $E(P) - M$. All vertices that were saturated remain saturated, and the endpoints of P become newly saturated.

The *symmetric difference* of two sets A and B is the set

$$A \triangle B = (A - B) \cup (B - A)$$

of elements in exactly one of the sets.

For graphs G and G' on the same vertex set the *symmmetric difference* is the graph $G \triangle G'$ with edge set $E(G) \triangle E(G')$.

Problem 3.1.3. Describe the components of the symmetric difference of two matchings of a graph.

Theorem 3.1.1 (Berge)**.** *A matching M in a graph G is maximum if and only if G has no M-augmenting path.*

Proof. It is clear that a maximum matching can have no augmenting path. We prove the other direction by proving the contrapositive statement: If M is not maximum, then G has an M-augmenting path.

Assume that M is a non-maximum matching. There is some bigger matching M'. The symmetric difference $M \triangle M'$ consists of cycles and paths that are alternating with repect to M. In a cycle, there are the same number of M and M' edges, so there must be some path with more M' edges than M edges. This is our M-augmenting path. □

Hall's Matching Theorem

A bipartite graph $G = G(X \cup Y)$ can have a perfect matching only if $|X| = |Y|$. When $|X| < |Y|$ the best we can hope for is an X-matching– a matching that saturates X. P. Hall gave a useful condition that characterises the existence of an X-matching

Theorem 3.1.2 (Hall's Theorem)**.** *A bipartite graph $G = G(X \cup Y)$ has an X-matching if and only if for every $S \subset X$ we have that $|N(S)| \geq |S|$.*

Proof. This condition is clearly necessary for the existence of the matching, as the matching defines a bijection from X to $N(X)$.

Sufficiency is harder; we prove the contrapositive. Assume that G has no X-matching. We show that there is a set $S \subset X$ such that $|N(S)| < |S|$.

Let M be a maximum matching of G. As G has no X-matching there is some unsaturated vertex $u \in X$. Let S and T be the set of vertices in X and Y respectively that are in M-alternating paths starting at u. Each vertex in T must be in an edge of M, or the M-alternating path it is in is M-augmenting, which is impossible as M is maximum. So M defines a bijection of T to $S - \{u\}$, showing that $|T| = |S| - 1$ and that $T \subset N(S)$. In fact $T = N(S)$, as any vertex $s \in S \setminus \{u\}$ is in an edge of M from some t in T, in an alternating path from u. Any other neighbour y it has extends this alternating path, so y is also in T.

Thus $T = N(S)$ and $|T| = |S| - 1 < |S|$ as needed. □

Problem 3.1.4. Show that any k-regular bipartite graph has a perfect matching.

Min-max Theorems

If I gave you a matching M of a graph G and told you it was a maximum matching, would you believe me? How do I prove it to you? How do I prove there is no bigger one.

When $G = G(X, Y)$ is bipartite, Hall's Theorem gives us a way, but it could be very inefficient. If my matching saturates s vertices in the partite set X of G, then I would need to show you the obstruction to a matching to any set of $s + 1$ vertices of X. (Exercise 3.1.3 in the text says you only have to do so for sets of $s + 1$ vertices containing these s vertices, which is much nicer, but still more work than we want.)

In this section we see a nicer way to certify that a matching of a bipartite graph is maximum.

For a set $Q \subset V(G)$ of vertices of a graph G we say that Q *covers* the set $\{uv \in E(G) \mid u \in Q\}$ of edges containing at least one vertex of Q. A set *covers* G if it covers $E(G)$.

Notice that if M is a matching of G, the at least one vertex in each edge of M must be in any cover Q of G. So $|Q| \geq |M|$ for any matching and any vertex cover of G.

Problem 3.1.5. Find a graph for which the maximum matching M is smaller than the minimum cover.

Theorem 3.1.3. *(König, Egerváry) For a bipartite graph G the size of the maximum matching of G is equal to the size of the minimum cover.*

Proof. Let Q be a minimum cover of bipartite $G = G(X, Y)$. We construct a matching M with $|M| = |Q|$, as $|Q| \geq |M|$ for any matching, this M is then maximum, and we are done.

Where we set

$$Q_X = Q \cap X \qquad \overline{Q}_X = \overline{Q} \cap X \qquad Q_Y = Q \cap Y \qquad \overline{Q}_Y = \overline{Q} \cap Y$$

let H be the subgraph of G induced on $Q_X \cup \overline{Q}_Y$, and H' be the subgraph induced on $Q_Y \cup \overline{Q}_X$. As Q is a cover, there are no edges from $\overline{Q}_x$ to $\overline{Q}_Y$, so all edges are in H or H' or from Q_X to Q_Y.

Let $S \subset Q_X$. If $|N_H(S)| < |S|$, then replacing S by $N_H(S)$ in Q, all edges covered by S are covered by $N_H(S)$ or Q_y, so we get a smaller cover of G. As this is impossible, we have $|N_H(S)| \geq |S|$ for all subsets S in $H \cap X$, and so by Hall's Condition, H contains a Q_X-matching. Similarily H' contains a Q_Y-matching. Together these give a matching of G of size $|Q|$.

□

Now, if I give you a matching that I say is maximum, I can prove it by giving you a cover of the same size. That's pretty nice.

The rest of the chapter looks at turning these proofs into algorithms to find a maximum matching of a graph, and then a generalisation, by Tutte, that gives for all graphs, not just bipartite ones, a Hall-like condition for deciding if a graph has a perfect matching. We skip this. We can't do everything!

Problems from the Text

3.1: 1,2,3, 8, 10, 11, 15, 16

4 Connectivity and Paths

4.1 Cuts and Connectivity

The connectivity of a graph is a measure of how hard (or easy) it is to disconnect it.

A *separating set* or *vertex cut* of a graph G is a set $S \subset V(G)$ such that $G-S$ has more components than G. The *connectivity* $\kappa(G)$ of a graph G is the size of the smallest set S of vertices such that $G - S$ is disconnected or is a single vertex. A graph G is k-connected if it has connectivity at least k. Connected graphs are 1-connected.

Problem 4.1.1. Show that $\kappa(G) \leq \delta(G)$.

Problem 4.1.2. Show that a graph has connectivity κ if and only if its smallest separating set has size κ, or it is $K_{\kappa+1}$.

The following are examples in the text.

Problem 4.1.3. What are $\kappa(K_{a,b})$ and $\kappa(Q_k)$?

Edge Connectivity

Closely related to the connectivity of a graph is its *edge connectivity*. Connectivity counts the number of vertices we must remove to kill conections, edge connectivity counts the number of edges we must remove.

A *disconnecting set* in a graph G is a set $F \subset E(G)$ such that $G \setminus F$ is disconnected. An *edge cut* of G is a disconnecting set of the form

$$[S, \overline{S}] = E(G) \cap \{s, t \mid s \in S, t \in \overline{s}\}.$$

All minimal disconnecting sets are edge cuts, but not all edge cuts are minimal disconnecting sets. A graph G is *k-edge-connected* if the smallest edge cut has at least k edges; its *edge connectivity* $\kappa'(G)$ is the size of its smallest edge cut. And edge cut consisting of one edge is a *cut-edge*.

The following is almost trivial. You have to be careful with cliques though– they are a separate case.

Problem 4.1.4. Show that $\kappa(G) \leq \kappa'(G) \leq \delta(G)$.

Problem 4.1.5. Find a graph G with $\kappa(G) < \kappa'(G)$ and one with $\kappa'(G) < \delta(G)$.

Theorem 4.1.1. *If G is 3-regular, then $\kappa(G) = \kappa'(G)$.*

Proof. Let S be a vertex cut of 3-regular G of size $\kappa = \kappa(G)$. We find an edge cut of size at most κ. Let A and B be components of $G - S$. Each vertex of

S must have at least one edge to each of A and B, so has only one edge to at least one of them. Put this edge in F for each $v \in S$, preferring the edge to A over the edge to B if there is only one of each. We claim F is an edge-cut whose removal kills all minimal AB-paths. Indeed, there are no 1-paths from A to B, and any 2-path must contain two edges of some s in S, and at least one of these edges was removed. A path of length $\ell \geq 3$ has second vertex s and $\ell - 1^{th}$ vertex s' in S. These have internal edges in the path that are not to A or B, so each have one edge each to A and B. Thus both have their edges to A in F, and so its removal kills the path. □

An edge cut $F = [S, \overline{S}]$ of a connected graph G is a minimal diconnecting set if S and $\overline{S}$ are both connected. So $G - F$ has two components. In this case F is called a *bond*.

Blocks

A graph without a *cut-vertex*– a vertex cut consisting of a single vertex– is either 2-connected, or K_1 or K_2. A *block* of a graph G is a connected subgraph that is maximally with respect to having no cut-vertices. (It can have cut-vertices of G, but they are not cut vertices of the block). The *block-cutpoint graph* of a connected graph G is the (bipartite) graph H whose vertices are the blocks and cut-vertices of G; a block is adjacent to the cut-vertices that it contains.

In Exercise 34 you show the following:

Fact 4.1.2. *The block-cutpoint graph of a connected graph is a tree whose leaves are blocks of G.*

To find the blocks of a connected graph one can first find a cut-vertex, remove it, breaking the tree into two smaller graphs, and recurse. Finding a cut-vertex, requires, in the simple approach, removing vertices one at a time, and check (with BFS maybe) if what remains is connected. Finding one this way takes $O(nm)$ basic operations, and so the whole process is $O(n^2m)$. Another spanning tree algorithm for graphs does it in time $O(m)$.

BFS was a breadth first search– we built a rooted spanning tree from r that was as wide as possible at each level. Now we will use a *depth first search* DFS. This builds a spanning tree from r that goes as deep as possible. Instead of visiting every possible edge of an active vertex before retiring it (to the set U) we visit only the first edge of the vertex and make the next vertex active. We don't retire the previous vertex though, we will have to 'backtrack' to it to look at its other neighbour later.

As with the BFS algorithm, we return a spanning tree T of G, but depending what we are interested in, we return other things as we go. In the following version we return the blocks of G as well.

Algorithm 4.1.3 (DFS Tree)**.**

Input: A connected graph G and a root vertex r.

Output: A spanning tree T rooted at r and the set $\mathscr{B}$ of blocks of G

Initialise: Tree $T := \{r\}$, active vertex $a := r$, blocks $\mathscr{B} := \emptyset$, used vertices and edges $U := \emptyset$.

Algorithm: While r not in U:

- If a has edge aw not in U: put it in U and if $w \notin T$ then add wa to T, and set $a := w$. Otherwise:
- If $a = r$ then put r in U. Otherwise:
- Where p is the predecessor of a in T, and T' is the subtree of T rooted at a,
  - If no vertex in T' has an edge to above p in T then put $V(T') \cup \{p\}$ in $\mathscr{B}$ and $V(T')$ in U.
  - Set $a := p$.

Problems from the Text

4.1: 1, 2, 3, 4, 5, 6, 7, 8, 10, 13, 19, 33, 34